



**UNIVERSIDAD
DE GRANADA**

Desarrollo de juego tipo *runner* con Arduino

Periféricos y Dispositivos de Interfaz Humana.

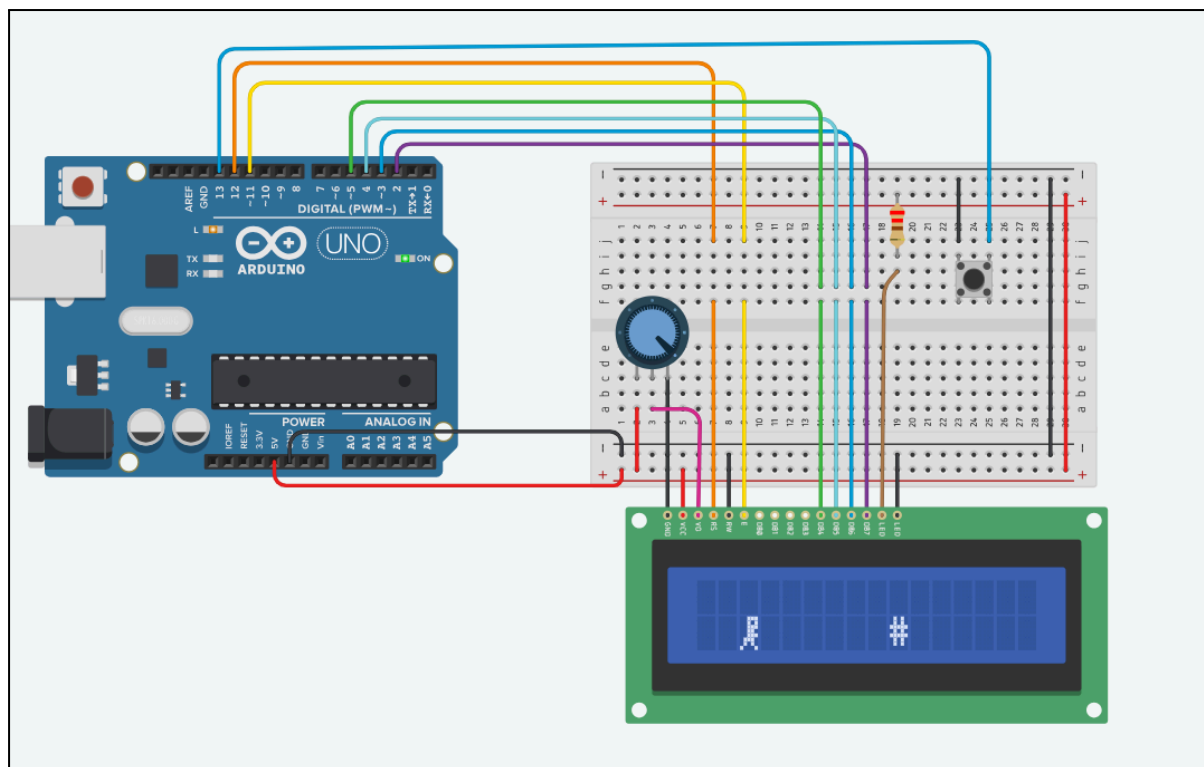
**Hecho por:
Miguel Moreno Murcia**

ÍNDICE

1. INTRODUCCIÓN.....	3
2. MONTAJE DEL HARDWARE.....	3
3. DESARROLLO DEL VIDEOJUEGO.....	5
4. REFERENCIAS.....	10

1. INTRODUCCIÓN

Para este proyecto se ha desarrollado un videojuego tipo runner, inspirado en el del dinosaurio de Google Chrome, implementado sobre una pantalla LCD 16x2 con Arduino. En las siguientes secciones se describe el montaje del hardware necesario, así como el desarrollo del software del juego.

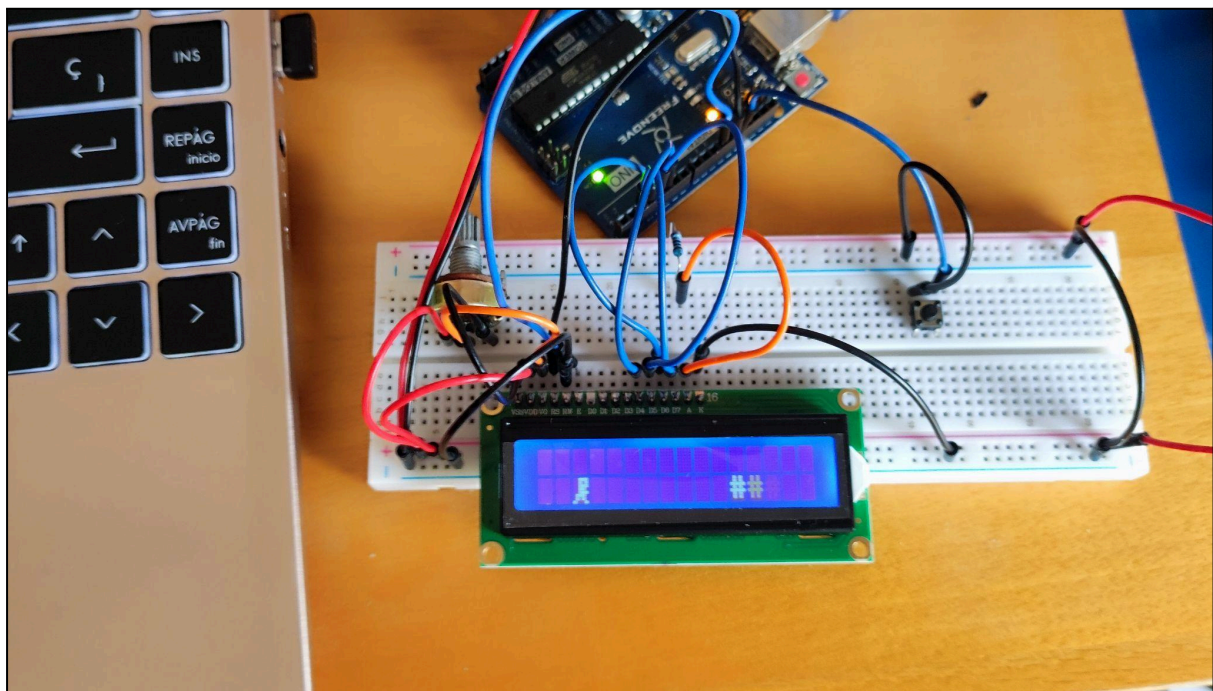
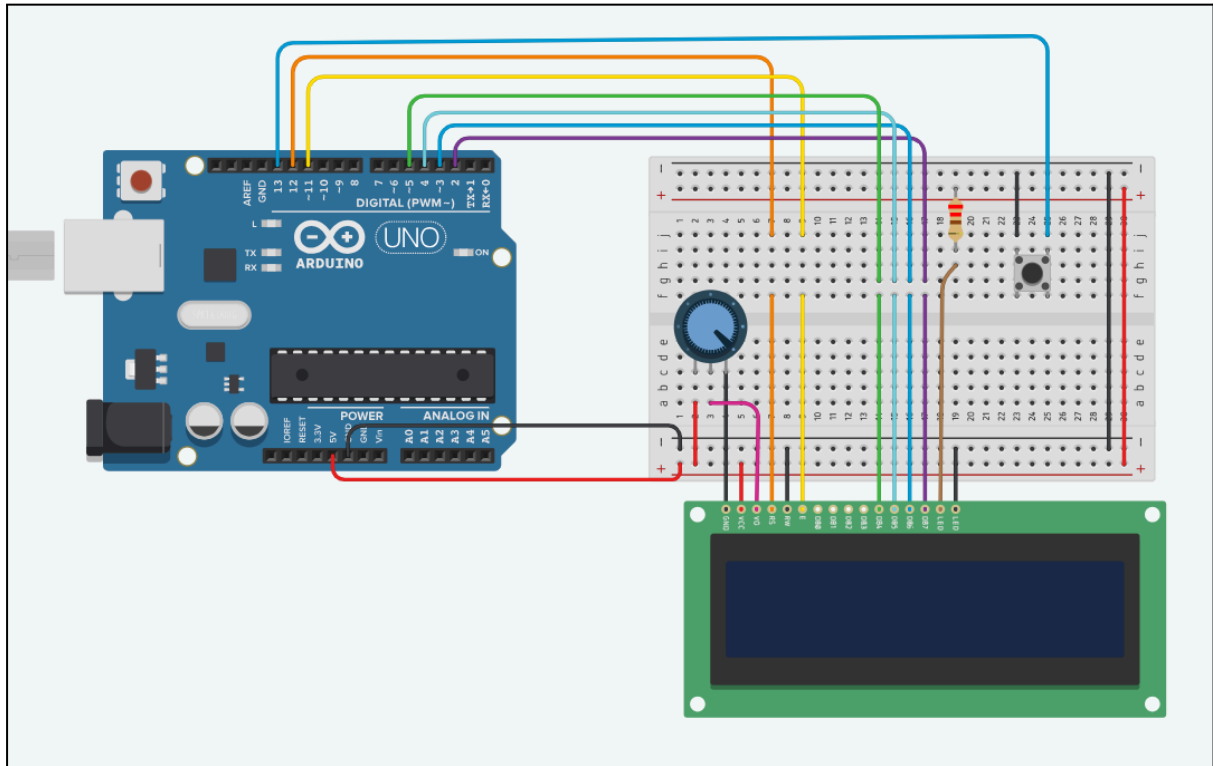


2. MONTAJE DEL HARDWARE

El sistema está basado en una placa Arduino UNO, una pantalla LCD 16x2 y un pulsador que se utiliza como entrada para controlar el salto del personaje dentro del juego.

La pantalla LCD permite representar los elementos del videojuego de forma textual y mediante caracteres personalizados. Está conectada en modo paralelo utilizando la librería LiquidCrystal, empleando seis pines digitales del Arduino para su control. En concreto, se han utilizado los pines 12, 11, 5, 4, 3 y 2, correspondientes a las señales de control y datos de la pantalla. Para montar la pantalla LCD he utilizado el siguiente [tutorial \[1\]](#).

El pulsador lo he conectado al pin digital 13 del Arduino. Este botón se utiliza como entrada principal del juego, permitiendo al jugador realizar el salto para evitar los obstáculos.



3. DESARROLLO DEL VIDEOJUEGO

Para crear el juego parecido al del dinosaurio necesitamos los siguientes elementos:

- Un jugador
- Obstáculos
- La lógica de salto y colisión con los obstáculos
- La lógica del inicio y fin de juego

Para empezar crearemos el jugador el cual situaremos en la fila inferior de la pantalla y en la columna 3 mediante las variables de posición *playerX* y *playerY*. Además para controlar el salto necesitamos una variable booleana que detecte si se puede saltar o si se está saltando (*salto*) y una variable que controle los ticks de juego que dura cada salto (*t_salto*).

Sobre el obstáculo haremos lo mismo asignándole una posición con las variables *obsX* y *obsY*. En este caso la posición inicial del obstáculo será en la fila inferior de la pantalla (la misma que la del jugador) y la última columna. Por último una variable para controlar si se choca contra un obstáculo *gameOver*.

La lógica del salto tiene dos puntos. El primero es leer el botón para realizar el salto. Si la variable *salto* es falsa, es decir, no estamos en un salto y pulsamos el botón, comienza el salto desplazando al jugador a la fila superior y poniendo a true la variable de salto. Cada tick de juego se mantiene al jugador en la fila superior y se resta 1 al contador de tiempo de salto.

La segunda parte ocurre cuando el contador de salto llega a cero. En ese momento, el jugador vuelve a la fila inferior y se pone a falso la variable de salto.

Para la mover el obstáculo por la pantalla en cada tick de juego restamos 1 a la posición x del obstáculo (*obsX*) para desplazarlo hacia el jugador. Cuando el obstáculo llega a la última columna de la pantalla se devuelve el obstáculo al otro extremo para que vuelva a moverse.

Comprobamos la colisión entre el jugador y el obstáculo comprobando si las posiciones son iguales. Si esto ocurre la condición de fin de juego se cumple y el juego termina. Cuando el juego termina se muestra en la pantalla que el juego ha terminado y las instrucciones para comenzar de nuevo. Para volver a jugar se debe pulsar el botón. Al pulsar el botón una vez se haya perdido se ejecuta la función

resetGame() que devuelve todas las variables a sus valores iniciales y comienza de nuevo el juego.

```
#include <LiquidCrystal.h>

// Crear objeto LCD
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// Pulsador en el pin 13
const int boton = 13;

// Variables del Jugador
int playerX = 2;
int playerY = 1;

// Variables sobre el salto
bool salto = false;
int t_salto = 0;

// Sprite de Jugador
byte player[8] = {
    B00111,
    B00101,
    B00111,
    B00110,
    B00111,
    B01110,
    B01010,
    B10001
};

// Variables de obstáculo
int obsX = 15;
int obsY = 1;

// Variable de fin de juego
bool gameOver = false;

void setup() {
```

```

// Inicializar el objeto LCD
lcd.begin(16, 2);

// Inicializar el pulsador
pinMode(boton, INPUT_PULLUP);

//Crear el Sprite
lcd.createChar(0, player);
}

void loop() {

    // Pantalla de fin de juego hasta pulsar botón
    if (gameOver) {
        lcd.setCursor(0, 0);
        lcd.print("GAME OVER      ");
        lcd.setCursor(0, 1);
        lcd.print("Pulsa boton      ");

        if (digitalRead(boton) == LOW) {
            resetGame();
        }

        delay(100);
        return;
    }

    // Movimiento del obstáculo
    obsX--;

    // Si se sale de la pantalla reaparecer
    if (obsX < 0) {
        obsX = 15;
    }

    // Leer botón (salto)
    if (digitalRead(boton) == LOW && !salto) {
        salto = true;
    }
}

```

```

    t_salto = 4;
}

// Si está saltando mover el jugador a la posición de salto
y decrementar el tiempo de salto en cada iteración
if (salto) {
    playerY = 0;
    t_salto--;

    // Si el tiempo de salto ha terminado devolver el jugador
    al suelo
    if (t_salto <= 0) {
        salto = false;
        playerY = 1;
    }
}

// Colisión
if (obsX == playerX && obsY == playerY) {
    gameOver = true;
}

// Limpiar LCD
lcd.clear();

// Dibujar Jugador
lcd.setCursor(playerX, playerY);
lcd.write(byte(0));

// Dibujar obstáculo
lcd.setCursor(obsX, obsY);
lcd.print("#");

delay(200);
}

// Devuelve a los valores iniciales para empezar el juego de

```



```
nuevo
void resetGame() {
    playerX = 2;
    playerY = 1;

    obsX = 15;
    obsY = 1;

    salto = false;
    t_salto = 0;

    gameOver = false;

    lcd.clear();
}
```

4. REFERENCIAS

[1]https://naylampmechatronics.com/blog/34_tutorial-lcd-conectando-tu-arduino-a-un-lcd1602-y-lcd2004.html